

FUNDAMENTOS FÍSICOS DE LA INFORMÁTICA

PRÁCTICA 3: APLICACIÓN DE LA REGLA DE LOS TRAPECIOS AL CÁLCULO DE MAGNITUDES FÍSICAS

Con ayuda de Matlab, vamos a implementar el método de los trapecios para calcular numéricamente el valor aproximado de una integral definida. Aprovecharemos la integración numérica para calcular diferencias de potencial y trabajos debidos a campos y fuerzas eléctricas que, en principio, pueden resultar difíciles de integrar.

En efecto, si lo que integramos es una función $E(x)$ que representa un campo eléctrico, el resultado de calcular la integral definida entre dos puntos del espacio $x=a$ y $x=b$ representa la diferencia de potencial entre dichos puntos. Por definición:

$$V(b) - V(a) = - \int_a^b \vec{E} \cdot d\vec{l} = - \int_{x=a}^{x=b} E(x) dx \quad (1)$$

siendo $\vec{E} = E(x)\vec{i}$ el vector campo eléctrico y $d\vec{l} = dx\vec{i}$ el desplazamiento.

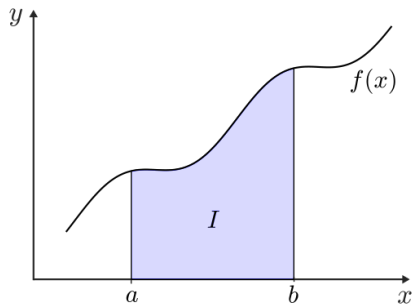
De la misma forma, si lo que integramos es una función $F(x)$ que representa la fuerza que utilizamos para desplazar una partícula en el espacio, el resultado de calcular la integral definida entre dos puntos del espacio $x=a$ y $x=b$ representa la variación de energía potencial de dicha partícula entre ambos puntos, es decir, el trabajo realizado por la fuerza para producir el desplazamiento. Por definición:

$$W_{a \rightarrow b} = U(a) - U(b) = \int_a^b \vec{F} \cdot d\vec{l} = \int_{x=a}^{x=b} F(x) dx \quad (2)$$

siendo $\vec{F} = F(x)\vec{i}$ el vector fuerza y $d\vec{l} = dx\vec{i}$ el desplazamiento.

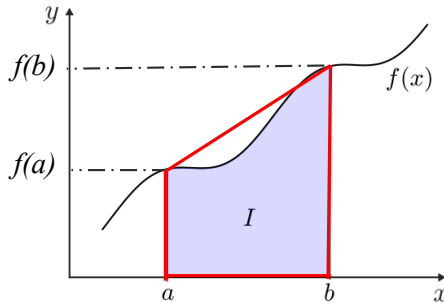
REGLA DE LOS TRAPECIOS

Cuando calculamos la integral definida I de una función $f(x)$ entre dos puntos a y b , realmente estamos calculando el área existente entre la curva $f(x)$ y el eje X de abscisas ente ambos puntos:



$$I = \int_a^b f(x)dx = \text{área bajo la curva}$$

De forma aproximada, podemos utilizar el método de los trapecios para calcular dicha área sustituyendo la función $f(x)$ entre a y b por una función lineal (color rojo) que une los puntos $(a, f(a))$ y $(b, f(b))$:



$$I \approx (b - a) \frac{f(a) + f(b)}{2}$$

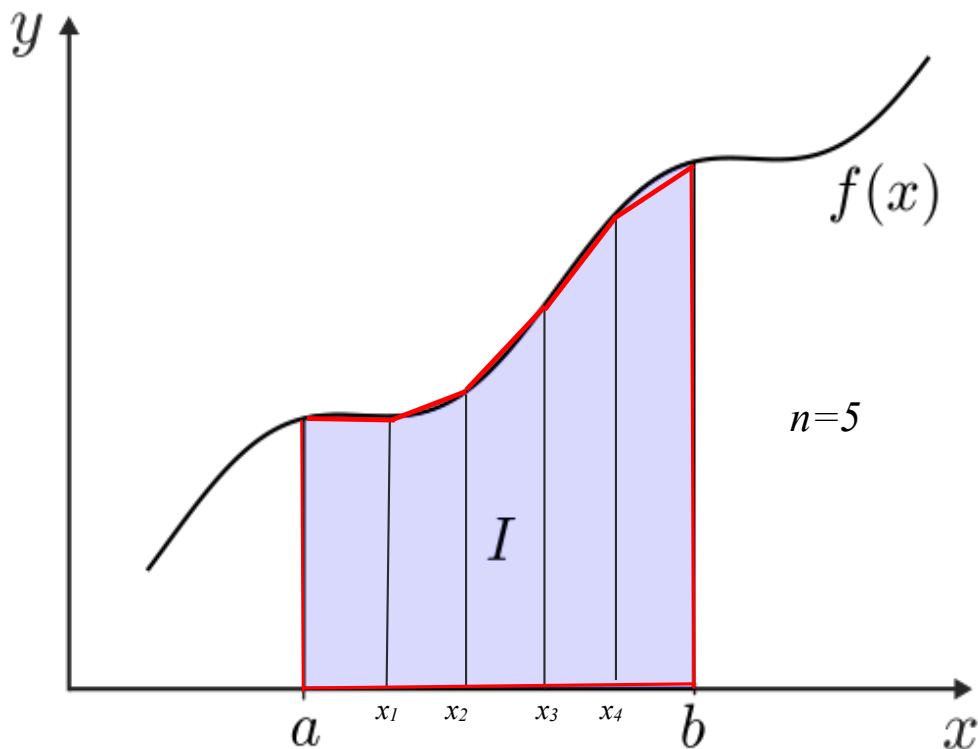
Matemáticamente, lo que hacemos es sustituir la función $f(x)$ por un polinomio de primer grado:

$$f_1(x) = f(a) + \frac{f(b) - f(a)}{b - a} (x - a)$$

Y proceder al cálculo de la integral:

$$I \approx \int_a^b f_1(x)dx = \int_a^b \left[f(a) + \frac{f(b) - f(a)}{b - a} (x - a) \right] dx = (b - a) \frac{f(a) + f(b)}{2}$$

Si queremos disminuir el error cometido en la aproximación, podemos subdividir el intervalo $[a, b]$ en n subintervalos y calcular el área como la suma de las áreas de los n trapecios:



Nuestra integral quedará entonces:

$$I = \int_a^b f(x)dx = \int_a^{x_1} f(x)dx + \int_{x_1}^{x_2} f(x)dx + \dots + \int_{x_{n-1}}^b f(x)dx$$

Si los $n+1$ puntos de corte están equiespaciados en el intervalo $[a, b]$, entonces la anchura de cada intervalo es constante e igual a $\Delta x = \frac{b-a}{n}$. Resolviendo cada una de las integrales mediante aproximación por polinomios de grado uno obtenemos:

$$I \approx \frac{b-a}{2n} \left[f(a) + 2 \sum_{i=1}^{n-1} f(x_i) + f(b) \right]$$

REGLA DEL TRAPECIO PARA MÚLTIPLES INTERVALOS

Si en vez de tener una función analítica $f(x)$ lo que tenemos es una serie de datos tabulados, nuestra expresión para múltiples intervalos varía ligeramente al no poder garantizar ahora el mismo tamaño para todos ellos:

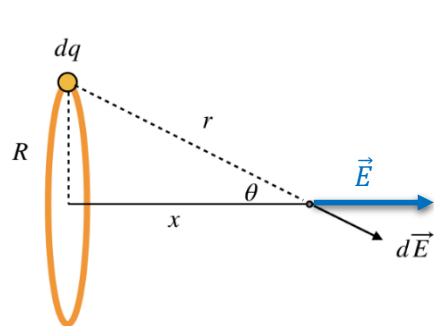
$$I \approx \frac{1}{2} \sum_{i=0}^{n-1} [\Delta x_i (y_i + y_{i+1})]$$

donde $\Delta x_i = (x_{i+1} - x_i)$ es la anchura de cada intervalo,

$y_i = f(x_i)$ corresponde a los valores tabulados de la función.

PROBLEMA BASE

Consideremos un anillo uniformemente cargado con una carga Q . El campo eléctrico resultante debido a la totalidad de la carga del anillo en un punto de su eje de simetría es:



$$\vec{E}(x) = \frac{k x}{(x^2 + R^2)^{3/2}} Q \vec{i}$$

k : constante de Coulomb
 R : radio del anillo

Y el potencial que crea a lo largo de su eje, podemos obtenerlo de forma exacta calculando la integral correspondiente:

$$V(x) = - \int E(x) dx = - \int \frac{k x}{(x^2 + R^2)^{3/2}} dx = \frac{k Q}{(x^2 + R^2)^{1/2}} \text{ con } V(\infty) = 0 \quad (3)$$

Así, la diferencia de potencial exacta entre dos puntos a y b viene dada por:

$$V(b) - V(a) = \frac{k Q}{(b^2 + R^2)^{1/2}} - \frac{k Q}{(a^2 + R^2)^{1/2}}$$

Lo que proponemos en esta práctica es utilizar la regla de los trapecios para calcular numéricamente la diferencia de potencial entre dos puntos $x=a$ y $x=b$. Es decir, supongamos que no conocemos la expresión exacta de $V(x)$ dada por la ecuación (3) y calculemos dicha diferencia, de forma aproximada, mediante cálculo numérico. Posteriormente compararemos los resultados.

IMPLEMENTACIÓN EN MATLAB

Vamos a crear un fichero de comandos (M-archivo con extensión *.m) escribiendo un nuevo *script* en la ventana del editor. En la primera línea escribimos la descripción del problema:

```
% REGLA DE LOS TRAPÉCIOS  
% integración numérica campo eléctrico para calcular ddp
```

A continuación, introducimos como datos del problema los puntos inicial $x=a$ y final $x=b$ para el cálculo de la *ddp* y el número de intervalos n en los que dividiremos la diferencia (menor número de intervalos mayor error de cálculo):

```
% introducir datos  
a=input('extremo inferior de integración:');  
b=input('extremo superior de integración:');  
n=input('número de intervalos:');
```

Definimos el vector x cuyos valores corresponden a cada uno de los puntos en los que dividimos el intervalo $[a,b]$, y sobre los que posteriormente calcularemos el campo eléctrico:

```
% Definimos el vector x:  
x = linspace(a,b,n+1);  
  
% Definimos el campo eléctrico creado por un anillo  
% de carga con  $kQ=60\text{Nm}^2/\text{C}$  y  $R=1\text{m}$  según ecuación (1)
```

```
campo = @(x) (60.*x)./(((x.^2)+1).^1.5);
```

```
% potencial (valor exacto según ecuación (2)):  
potencial = @(x) 60./(((x.^2)+1).^0.5);
```

```
%diferencia de potencial exacta  
ddp_exacta=potencial(b)-potencial(a);  
ddp_exacta
```

```
% función que vamos a integrar  
y = campo(x);
```

```
% Realizamos la integración por trapecios
```

$v_{\text{aprox}} = -\text{trapz}(x,y);$ % trapz es una función de matlab que calcula por el método de trapecios el área bajo la curva $y=\text{campo}(x)$

el signo menos se debe a que, por definición, $V(b) - V(a) = - \int_a^b \vec{E} \cdot d\vec{l}$.

```
% Dibujamos el área bajo la curva
fig = figure; fig.Color = "w";
grid on % la cuadrícula
plot(linspace(a,b,200),campo(linspace(a,b,200)),"r","linewidth",2); % la función campo eléctrico
hold on
plot(linspace(a,b,200),potencial(linspace(a,b,200)),"g","linewidth",2); % el potencial (exacto)
hold on
xlabel('x') % etiquetamos el eje X
ylabel('E, V') % etiquetamos el eje Y
title("Integración por trapecios"); % ponemos título

% Dibujamos los trapecios de integración y rellenamos el
área uno a uno
    for i = 1:numel(x)-1
area(x(i:i+1),y(i:i+1),"FaceColor","b","FaceAlpha",0.15)
% la función de matlab 'area' rellena áreas bajo curvas
    end
text(x(1),-0.1,"a","FontSize",15)
text(x(end),-0.1,"b","FontSize",15)
hold off

% Ponemos la leyenda
s1 = sprintf("$area$, n = %d",n);
legend("$campo$", "$potencial$",
s1,"FontSize",11,"Interpreter","latex")

% Mostramos el valor numérico (aproximado) de la
integral (la ddp entra a y b)
xt=(a+b)/2; yt=(potencial(b)+potencial(a))/2;
% definimos las coordenadas x e y dónde aparecerá el
texto
```

```
text(xt,yt, "$ddp_{ba}\approx$ " + num2str(ddp_aprox,5) +
"$ V (aprox)$", "Interpreter","Latex","FontSize",14) %
num2str convierte números en array de caracteres
```

Guardamos el fichero de comandos: *practica3.m* y a continuación lo ejecutamos desde la ventana de comandos con para comprobar su correcto funcionamiento:

1. Tomar los valores exactos del potencial procedentes de la ecuación (2) para calcular la ddp en los siguientes intervalos con $kQ=60 \text{ Nm}^2/\text{C}$:

caso	a (m)	b (m)
i	-5	+5
ii	-1	+5
iii	0	5

2. Utilizar la regla de los trapecios para calcular las ddp en los casos ii y iii con $n=5, 10, 50$, y 100 pasos. Comparar el resultado respecto al apartado anterior ¿Cómo varía el error cometido al aumentar el número de intervalos?
3. Ejecutar el programa para el caso $a=1 \text{ m}$ y $b=10 \text{ m}$ con $n=5, 10, 50$ y 100 pasos para el campo eléctrico $E(x) = \frac{60}{x^2}$ con x expresado en metros ¿Cuál es ahora el valor exacto del potencial?

PROBLEMA PROPUESTO 1: DDP CON DATOS TABULADOS

Se propone aplicar la regla de los trapecios al cálculo de la diferencia de potencial entre los puntos $x=0$ y $x=2 \text{ m}$ debida a la presencia de un campo eléctrico cuyos valores tabulados (en el SI de unidades), se encuentran recogidos en *tabulatedData.m*

```
function T = tabulatedData
    % datos de un campo desconocido (aquí son la función
    seno)
    col1 = [0 0.05 0.1 0.15 0.2 0.25 0.3 0.35 0.4 0.45
0.5 0.55 0.6 0.65 0.7 0.75 0.8 0.85 0.9 0.95 1 1.1 1.2
1.3 1.4 1.5 1.6 1.7 1.8 1.9 2]';
    col2 = [0.904673 0.732028 0.50591 0.252999 -0.000003
-0.228929 -0.414208 -0.542303 -0.606422 -0.606531 -
0.606531 -0.54871 -0.443995 -0.306847 -0.153448 0.000005
0.138855 0.251233 0.328925 0.367814 0.367879 0.269295
```

```
0.093069 -0.084222 -0.199504 -0.22313 -0.163335 -
0.056448 0.051084 0.121006 0.135335]';
    T = table(col1,col2,'VariableNames',{'x','y'});
end
```

Observar que los intervalos en x para los datos conocidos de campo eléctrico (y) van variando. Escribimos el código:

```
%leemos los datos de la tabla
T = tabulatedData;
% dibujamos los datos
plot(T.x,T.y,"ko")
xlabel("x")
ylabel("y")
grid on
hold on
set(gca,"XAxisLocation","origin","XTick",0:0.25:3)
% identificamos nuestros datos x,y
x = T.x;
y = T.y;
% cálculo de la integral I (la ddp)
I = 0; % inicializamos una variable
n = numel(x); % número de puntos del dataset
for i = 1:(n-1)
    I = I + (x(i+1) - x(i))*0.5*(y(i) + y(i+1));

area(x(i:i+1),y(i:i+1),"FaceColor","y","FaceAlpha",0.34)
end
hold off
% Mostrar el resultado
text(x(end)-2,2,"$ddp_{ab}\approx$ " +
num2str(I,4)+"$ V$", "Interpreter","Latex","FontSize",14)
fprintf("ddp = %2.5f",I)
```

1. Crea un conjunto de datos tabulados equidistantes de anchura $\Delta x = 0,1 \text{ m}$ para el campo eléctrico $E(x) = 3 \sin(2\pi x - 0,1)$. Calcula la ddp entre $x=-1 \text{ m}$ y $x=2 \text{ m}$.

PROBLEMA PROPUESTO 2: CÁLCULO DE TRABAJO

Se propone aplicar la regla de los trapecios al cálculo del trabajo (según ecuación (2)) que hay que realizar para trasladar una carga de 3 C desde el punto $x=0.1$ m hasta el punto $x=10$ m, en presencia de un campo eléctrico $E(x) = xe^{-0.8x} + 0.2$ (coordenadas x expresadas en metros). Hacerlo para valores $n=5, 10, 50$, y 100 pasos.

1. ¿Es trabajo a favor o en contra del campo eléctrico? Razona tu respuesta.
2. Incluir en la implementación Matlab una línea de código en la que se calcule la variación de energía cinética y la energía potencial de la partícula y se saque por pantalla.

ANEXO: FUNCIONES ANÓNIMAS

Las funciones anónimas en Matlab se definen en la ventana de comandos o en un archivo-m script y están disponibles sólo hasta que se limpia la zona de trabajo.

Las funciones anónimas acceden a las variables del *workspace* en el que son definidas creando una copia de las variables que están utilizando. El valor de dichas variables ya no se actualiza; sólo los argumentos pueden cambiar de valor. De esta forma, las funciones anónimas pueden ejecutarse a través del handle (@) en otros espacios de trabajo.

Uso de las funciones anónimas

Veamos un ejemplo:

```
ln = @(x) log(x)
```

ln = function_handle with value:

```
@(x)log(x)
```

```
ln(12)
```

```
ans = 2.4849
```

El símbolo @ alerta a MATLAB que **ln** es una función.

Inmediatamente después del símbolo @, se menciona la entrada a la función.

Finalmente, se define la función.

El nombre de función aparece en la ventana de variables, mencionada como una *function_handle* (manipulador de función).

Una vez que se limpia el área de trabajo, la función anónima ya no existe más.

```
clear all  
ln(12)
```

Unrecognized function or variable 'ln'.

Las funciones anónimas se pueden guardar como archivos .mat, y se pueden restaurar con el comando **load**.

```
ln = @(x) log(x)
```

ln = *function_handle with value:*

```
@(x)log(x)
```

```
save('milogaritmo.mat','ln')  
clear all  
ln(12)
```

Unrecognized function or variable 'ln'.

```
load('milogaritmo.mat')  
ln(12)
```

```
ans = 2.4849
```

Múltiples funciones anónimas

Podemos usar las funciones anónimas dentro de otra función anónima en nuestro código en matlab. Esto es especialmente útil en el caso de que se desee pasar diferentes parámetros a una función que se esté evaluando dentro de un rango de valores.

Ejemplo:

$$h(k) = \int_0^1 (kx + 2)dx$$

```
g = @(k) (integral (@(x) (k*x+2),0,1);  
g(6)
```

ans = 5